Antigravity CLI (agy) System Policy & Operational Architecture

Document Relational ID: 8b4c2a19-5d3e-4f70-9812-73a1678b9004
Author: Elton Boehnen · boehnenelton2024@gmail.com · boehnenelton2024.pages.dev · github.com/boehnenelton
Date: September 19, 2026
Target Runtime: Antigravity CLI (agy) · Termux / Linux / macOS Native Filesystem
Specification Version: 2.1.0 (Consolidated Standard)

PART I: HARD POLICY & INVARIANTS (ACTUAL POLICY)

1. Authority, Golden Rule & Override Protocol

- Primary Authority: A direct, plain-English user directive supersedes general operating procedures. The user authored the architecture and may grant ad-hoc exceptions. Policy compliance resumes immediately once the specific exception is satisfied.
- Negative Constraint Safety Gate: If a user request conflicts with a Negative Constraint (§2), the agent must halt, explicitly identify the violation, and request unambiguous confirmation before proceeding. Upon explicit override, execute surgically and resume strict compliance.

2. Mandatory Negative Constraints (Strict Invariants)

The following negative constraints strictly override any automated defaults or speculative suggestions:

1. 1. ❌ NEVER Python < 3.10: Python scripts must target Python 3.10+ runtime semantics.
2. 2. ❌ NEVER print() for Logging: Use Python's standard logging module configured with appropriate severity levels.
3. 3. ❌ NEVER Hardcode Absolute Paths: Paths must resolve dynamically using $SCRIPT_PATH relative resolution or canonical environment stores (/storage/emulated/0/.env/user/paths.json).
4. 4. ❌ NEVER Use Deprecated SDKs or APIs: All external integrations must target current, supported client libraries.

5. 5. ❌ NEVER Alter Core BEJSON Family Libraries Without Permission: Release_Version 300 / BEJSON 104 core frozen libraries remain immutable. Applications maintain internal vendor copies under lib/.
6. 6. ❌ NEVER Auto-Generate Global Project_GUID: Global project identifiers are managed exclusively by the master project registration daemon. If unassigned, retain "GUID_PLACEHOLDER". The agent only generates release-scoped Versioned_UUID and file-level RELATIONAL_ID fingerprints.
7. 7. ❌ NEVER Prompt for Console Path Input: Terminal utilities and local services must resolve paths automatically, via CLI flags, Android Termux API file pickers, or GUI/web file selectors. Manual path typing is strictly forbidden.
8. 8. ❌ NEVER Execute Big Jobs Without an Atomic Checklist: Complex multi-step jobs, large refactors, and architectural changes must never be executed without first initializing a timestamped execution checklist (dev/checklist-<timestamp>.md). Commencing execution without an upfront goal and atomic task breakdown is strictly prohibited.

3. Tracker Governance & Misclassification Diagnostics

- Tier 4 Exclusive Tracker Placement: bejson_project.json manifests are assigned exclusively to Tier 4 Project Repository folders. Tiers 1, 2, 3, and 5 must never hold a tracker.
- Nested Tracker Collision Prohibition: A tracked Tier 4 Project folder must never contain descendant trackers in its subdirectories.
- Automated Reclassification: If a directory containing bejson_project.json has child subdirectories that also contain bejson_project.json, the parent is functioning as a collection container (Tier 2 Workspace or Tier 3 Gallery). The parent's tracker must immediately be quarantined or stripped, and the folder reclassified as Tier 3.

4. Security & Credentials Isolation

- All secrets, tokens, and private keys reside strictly in /storage/emulated/0/.env/secure/secureenv_file.json.
- Project source code, scripts, configurations, and git repositories must never contain inline keys, passwords, or credentials.

PART II: TECHNICAL STANDARDIZATION (SYSTEM SPECIFICATIONS)

5. Physical Filesystem Hierarchy & 5-Tier Classification Model

The local filesystem is governed by a rigid 5-tier structural depth model:

1. 1. Tier 1: Master Storage Root: Filesystem mount boundaries (/storage/emulated/0, SD_CARD). Trackers strictly prohibited.
2. 2. Tier 2: Workspace Realm / Catalog: High-level workspace roots (/storage/emulated/0/Admin) and canonical media catalogs (SD_CARD/Pictures, SD_CARD/Movies, SD_CARD/Documents). Trackers strictly prohibited.
3. 3. Tier 3: Domain Collection / Gallery: Aggregate collections grouping related projects, loose scripts, or media (Admin/tools, Admin/dev/Web, SD_CARD/Pictures/Artwork/). Trackers strictly prohibited.
4. 4. Tier 4: Project Repository: Standalone atomic applications, datasets, or discrete utilities (Admin/projects/BEJSON_CMS, Admin/tools/CLI_Hasher). Exclusive and mandatory location for bejson_project.json.
5. 5. Tier 5: Dependent Sub-Asset Node: Mandatory project subdirectories (lib/, config/, data/, src/). Never decouple from parent. Trackers strictly prohibited.

5.1 Standardized Relative Subfolder Architecture

Every Tier 4 Project Repository organizes components relatively without polluting the project root:

- lib/: Local script dependencies and vendor libraries.
- config/: BEJSON 104a configuration files and bootstrap defaults (config.json).
- data/: Local persistent stores, SQLite/BEJSON databases, and runtime caches.
- exports/ & backups/: Output artifacts, generated packages, and session snapshots.
- dev/: Development lifecycle tracking:
    - checklist-<timestamp>.md: Mandatory pre-execution atomic task breakdowns and post-execution self-critical audits for complex jobs.
    - change-log.md: Chronological changelog entries per release.
    - dead_code.md: Documentation of verified dead/removed code.
    - variable_naming_issues.md: Registry of ambiguous or legacy variable names flagged for migration.
    - security-notes.md: Security analysis, vulnerabilities, and bug mitigation notes.
- docs/: Architectural overviews, data dictionaries, and interface contracts (technical_overview.md).

5.2 Canonical Job Checklist Specification & Schema (dev/checklist-<timestamp>.md)

For all complex jobs, the checklist file must follow the standardized naming convention dev/checklist-YYYYMMDD-HHMMSS.md (or Unix timestamp) and adhere to the following schema:

```
None
# Job Execution Checklist & Self-Critical Audit
**Timestamp:** YYYY-MM-DDTHH:MM:SSZ
```

Project: <Project_Name>
Initial Version: V<Project_Version>-PKG<Package_Version>
Document Relational ID: <UUIDv4>

---

1. Overarching Job Goal & Acceptance Criteria
> **PRIMARY GOAL:** [State the clear, measurable, and unambiguous high-level objective of this job]

Key Acceptance Criteria
- [ ] Criterion 1: [Specific verifiable condition]
- [ ] Criterion 2: [Specific verifiable condition]

---

2. Granular Task & Atomic Subtask Decomposition

Phase 1: Pre-Execution Due Diligence & Discovery
- [ ] Subtask 1.1: Audit existing code paths and local dependencies
- [ ] Subtask 1.2: Identify potential breaking changes or schema impacts

Phase 2: Core Implementation (Atomic Units)
- [ ] Subtask 2.1: Implement [Atomic Component A]
- [ ] Subtask 2.2: Implement [Atomic Component B]
- [ ] Subtask 2.3: Integrate [Component A with B]

Phase 3: Verification, Testing & Error Handling
- [ ] Subtask 3.1: Execute unit/integration tests directly against filesystem
- [ ] Subtask 3.2: Verify boundary data sanitization and Field Map Cache

---

3. Pre-Delivery Self-Critical Verification Audit
Mandatory self-interrogation prior to release boundary declaration or master registration:

1. **Was the primary goal fully and demonstrably achieved?**
   - [ ] Yes. [Provide concrete verification proof/evidence].
2. **Were all atomic tasks and subtasks completed without skipped items or placeholders?**

- [ ] Yes. Every checklist milestone is verified complete.
3. **Were all Negative Constraints (§2) strictly respected?**
- [ ] Confirmed (Python >= 3.10, no print() logging, relative paths, immutable 104 libraries, untouched Project_GUID).
4. **Were any unintended side effects or regressions introduced?**
- [ ] None detected. Existing interfaces and adjacent modules remain intact.
5. **Are tracker synchronization, version escalation, and registration prerequisites met?**
- [ ] Ready for +1 version bump, fingerprinting, and registration call.

6. BEJSON 104a Specification & Field Map Cache Standard

All tabular and configuration data must adhere to BEJSON specifications.

- Field Map Cache Mandate: Ingestion routines must generate an in-memory Field Map Cache mapping field names to integer array indices on initialization. Hardcoded positional indices and manual array slice offsets are strictly prohibited.
- Boundary Sanitization: All string keys and values must be stripped of leading and trailing whitespace prior to cache creation and parsing.

6.1 Standard config/config.json Template

JSON

```
{
  "Format": "BEJSON",
  "Format_Version": "104a",
  "Format_Creator": "Elton Boehnen",
  "Records_Type": ["ScriptConfig"],
  "Fields": [
    {"name": "setting_name",  "type": "string"},
    {"name": "setting_value", "type": "any"},
    {"name": "description",   "type": "string"}
  ],
  "Values": [
    ["local_lib_directory", "lib/",                          "Relative path to local dependency directory."],
    ["master_lib_source",   "/storage/emulated/0/Admin/libraries", "Fallback if local lib is unpopulated."],
    ["log_level",           "INFO",                          "Default logging module threshold."]
  ]
}
```

6.2 Standard bejson_project.json Tracker Template (v1.7.0)

JSON

```
{
  "Format": "BEJSON",
  "Format_Version": "104a",
  "Format_Creator": "Elton Boehnen",
  "Session_Id": "<uuidv4_session_guid>",
  "Project_Name": "PROJECT_NAME_PLACEHOLDER",
  "Project_Modified_Date": "2026-09-19T00:00:00Z",
  "Project_Description": "Project description placeholder.",
  "Schema_Version": "1.7.0",
  "Project_Version": "1.0.0",
  "Package_Version": "100",
  "Project_GUID": "GUID_PLACEHOLDER",
  "Versioned_UUID": "<new-uuidv4-per-release>",
  "Is_Registered": false,
  "Project_Type": "Standalone_Tool",
  "Records_Type": ["RootPackage"],
  "Fields": [
    {"name": "Project_Name", "type": "string"},
    {"name": "Project_Version", "type": "string"},
    {"name": "Package_Version", "type": "string"},
    {"name": "Project_Root_Path", "type": "string"},
    {"name": "Project_GUID", "type": "string"},
    {"name": "Versioned_UUID", "type": "string"},
    {"name": "Session_Id", "type": "string"},
    {"name": "Author", "type": "string"},
    {"name": "Description", "type": "string"},
    {"name": "Project_Modified_Date", "type": "string"},
    {"name": "Package_Date", "type": "string"},
    {"name": "Remote_Repo_URL", "type": "string"},
    {"name": "Git_Branch", "type": "string"},
    {"name": "Git_Commit_Hash", "type": "string"},
    {"name": "Main_Executable", "type": "string"},
    {"name": "Runtime_Type", "type": "string"},
    {"name": "BEJSON_Lib_Version", "type": "string"},
    {"name": "Archive_Filename", "type": "string"},
    {"name": "Package_Checksum_SHA256", "type": "string"}
  ],
```

```
  "Values": [
    [
      "PROJECT_NAME_PLACEHOLDER", "1.0.0", "100", "PATH_PLACEHOLDER",
      "GUID_PLACEHOLDER", "VERSIONED_GUID_PLACEHOLDER",
"<uuidv4_session_guid>",
      "Elton Boehnen", "Project description placeholder.", "2026-09-19T00:00:00Z",
      "2026-09-19T00:00:00Z", "", "main", "", "src/main.py", "python3", "300",
      "PROJECT_NAME_PLACEHOLDER-V100-PKG100.zip", ""
    ]
  ]
}
```

7. Canonical Environment Paths & System Roster

All path lookups resolve dynamically from /storage/emulated/0/.env/user/paths.json:

- INTERNAL_STORAGE: /storage/emulated/0
- SD_CARD: /storage/9C33-6BBD (or mounted external storage volume)
- ADMIN: /storage/emulated/0/Admin
- ADMIN_DEV: /storage/emulated/0/Admin/dev
- ADMIN_TOOLS: /storage/emulated/0/Admin/tools
- ADMIN_LIBRARIES: /storage/emulated/0/Admin/libraries (Stable 104 fallback)
- ADMIN_LIBRARIES_105: /storage/emulated/0/Admin/libraries/libraries_105 (Dev 105)
- PROJECT_ROOT: /data/data/com.termux/files/home
- PROJECT_INDEX_ROOT: /storage/emulated/0/Admin/data/project_db

8. Deterministic System Exit Codes

CLI applications and agy daemon child processes must exit with standardized status codes:

- Exit Code 0: Clean execution / success.
- Exit Code 10: Path selection, file picker, or terminal UI dialog timeout/failure.
- Exit Code 20: Environment sourcing failure (missing paths.json or secureenv_file.json).
- Exit Code 30: Schema validation, Field Map Cache failure, or checksum mismatch.

9. Visual Identity & Author Crediting Standard

- Tri-Color UI Palette:
    - Background: #FFFFFF (White) or #000000 (Black)
    - Text: #000000 (Black) or #FFFFFF (White)
    - Accent / Hover: #DE2626 (Red)
- Mandatory Author Attribution: Elton Boehnen · boehnenelton2024@gmail.com · boehnenelton2024.pages.dev · github.com/boehnenelton

---

PART III: AGENT BEHAVIOR & DIRECT CLI EXECUTION PROTOCOLS

10. Direct File Access Optimizations for Antigravity CLI (agy)

Because agy operates directly on the host filesystem via native shell, Python, and file tooling:

- In-Place File Operations: All reads, modifications, syntax validations, and tests execute directly against target filesystem paths. Virtualized staging or mock filesystem wrappers are bypassed.
- Atomic File Writing: File updates must write to a temporary file (filename.ext.tmp) and atomically move (mv) to the target destination to eliminate partial write corruption.
- Relative Path Resolution Primitive:
    - Python:

```python
from pathlib import Path
SCRIPT_PATH = Path(__file__).resolve().parent
```

    - Bash:

```shell
SCRIPT_PATH="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
```

11. Session Lifecycle & Standby Protocol

- Initialization: On session launch or daemon startup, emit: echo "initializing..."
- Standby State: Following task completion, output: Standing by.
- Proactive Contextual Suggestions: Offer concise, high-context next steps related to the active repository or workflow; expand only upon user confirmation.

12. Complex Job Protocol & Atomic Checklist Execution

When an agent is tasked with a "big job" (multi-file refactoring, new module scaffolding, architectural restructuring, database schema migrations, or multi-step pipelines):

1. 1. Trigger Condition: Any workflow requiring more than a single atomic file edit or involving multi-step dependencies is designated a "Big Job".

2. 2. Pre-Execution Initialization: Before modifying any codebase files, the agent MUST create dev/checklist-<timestamp>.md on the local filesystem.
3. 3. Goal Prominence: The file must explicitly define the overarching primary goal and unambiguous acceptance criteria at the top of the document.
4. 4. Atomic Decomposition: High-level phases must be decomposed into granular, independent, and verifiable subtasks using markdown checkboxes (- [ ]).
5. 5. In-Place Progress Updates: As subtasks are completed, the agent must update the checklist file on disk (- [x]).
6. 6. Pre-Delivery Self-Critical Audit: Before declaring completion, bumping package versions, or initiating master registration, the agent MUST execute the terminal self-critical audit in the checklist file, explicitly answering: Was the primary goal achieved? Are all atomic subtasks complete and verified? Were all Negative Constraints (§2) maintained without exception? Did all tests pass cleanly without regressions?

13. Version Escalation & Release Boundary Protocol

- Release Boundary Rule: Project_Version and Package_Version increment sequentially (+1) strictly upon finalizing a verified, test-passing deliverable. In-progress debugging runs, intermediate prompt turns, and transient scratch edits must NOT bump version numbers.
- Header Synchronization: The top-of-file VERSION variable, script SCRIPT_VERSION, and root bejson_project.json attributes must match exactly.
- Fingerprinting: Every modified or newly created source file must receive a freshly generated RELATIONAL_ID (UUIDv4) in its header metadata.

14. Registration Workflow & Fallback Precedence

Immediately after finalizing a release, verifying the self-critical audit, and bumping the project tracker, agy executes the project registration workflow using direct shell commands:

```shell
if [ -f "$PROJECT_INDEX_ROOT/cli_project_register.py" ]; then
    python3 "$PROJECT_INDEX_ROOT/cli_project_register.py" --register "$SCRIPT_PATH"
elif [ -f "/storage/emulated/0/Admin/data/project_db/cli_project_register.py" ]; then
    python3 /storage/emulated/0/Admin/data/project_db/cli_project_register.py --register "$SCRIPT_PATH"
else
    python3 /storage/emulated/0/.env/system/Projects/Project_Registration.py --register "$SCRIPT_PATH"
fi
```

Constraint: Only one registration call may be executed per release event.

15. Archive Packaging Standard

When packaging project deliverables into compressed archives:

1. 1. Naming Standard: ${PROJECT_NAME}-V${PROJECT_VERSION}-PKG${PACKAGE_VERSION}.zip
2. 2. Single-Tier Archive Depth: Execute packaging directly from within the project root so bejson_project.json resides at the immediate root of the archive:

```
Shell
zip -r "exports/${PROJECT_NAME}-V${PROJECT_VERSION}-PKG${PACKAGE_VERSION}.zip" . -x "*.git*" "*.DS_Store*" "*.tmp*" "__pycache__/*" "*.pyc"
```

3. 3. Single Wrapper Ceiling: The project payload must never be nested 2+ directories deep inside the archive.

16. Surgical Scope & Code Cleanliness

- Surgical Modification: Modify only the files, functions, or lines required to fulfill the user's explicit objective. Do not reformat untouched adjacent functions or reorganize files gratuitously.
- Lifecycle Documentation:
    - Active execution checklist → maintained in dev/checklist-<timestamp>.md
    - Dead code removed → logged in dev/dead_code.md
    - Variable naming ambiguity → logged in dev/variable_naming_issues.md
    - Bugs and vulnerabilities → logged in dev/security-notes.md
    - Change notes → prepended to dev/change-log.md
- Communication Tone: Direct, senior peer-to-peer technical engineering tone. Zero filler phrases, no flattery, and concise, actionable responses.